

Automated PCB Defect Detection System

Technical Documentation

Industrial AI Inspection Project

December 29, 2025

Abstract

This document presents a comprehensive technical overview of an end-to-end Automated PCB Defect Detection System utilizing deep learning and computer vision techniques. The system achieves 98.63% validation accuracy through a multi-module pipeline combining image subtraction, contour detection, EfficientNet-B0 classification, and an interactive web interface. This documentation covers dataset preparation, model architecture, training procedures, deployment strategy, and evaluation metrics across 6 defect categories.

Contents

1	Introduction	4
1.1	Project Overview	4
1.2	System Architecture	4
1.3	Defect Categories	4
2	Milestone 1: Dataset Preparation & Image Processing	6
2.1	Module 1: Dataset Setup and Image Subtraction	6
2.1.1	Objectives	6
2.1.2	Technical Implementation	6
2.1.3	Results	7
2.2	Module 2: Contour Detection & ROI Extraction	7
2.2.1	Objectives	7
2.2.2	Technical Implementation	8
2.2.3	Results	9
3	Milestone 2: Model Training & Evaluation	11
3.1	Module 3: EfficientNet-B0 Training	11
3.1.1	Model Architecture	11
3.1.2	Training Configuration	11
3.1.3	Data Augmentation	12
3.1.4	Training Results	12
3.1.5	Evaluation Metrics	12
3.2	Module 4: Inference Pipeline	13
3.2.1	Ultimate Inference Architecture	13
3.2.2	Implementation Highlights	13
3.2.3	Inference Performance	14
4	Milestone 3: Frontend & Backend Integration	15
4.1	Module 5: Streamlit Web Interface	15
4.1.1	UI Design Philosophy	15
4.1.2	Key Features	15
4.2	Module 6: Backend Pipeline	16
4.2.1	Architecture	16
4.2.2	Model Caching Strategy	17
4.2.3	Error Handling	17
5	Milestone 4: Finalization & Delivery	18
5.1	Module 7: Performance Analytics & Export	18
5.1.1	Analytics Dashboard	18

5.1.2	Key Metrics Tracked	18
5.1.3	Export Capabilities	19
5.1.4	System Optimization	19
5.2	Batch Evaluation System	19
5.2.1	Automated Testing Pipeline	19
5.2.2	Evaluation Results	20
6	Deployment Guide	21
6.1	System Requirements	21
6.1.1	Hardware	21
6.1.2	Software	21
6.2	Installation Steps	21
6.2.1	Clone Repository	21
6.2.2	Install Dependencies	21
6.2.3	Download Pre-trained Model	21
6.2.4	Configure Paths	22
6.3	Running the Application	22
6.3.1	Web Interface	22
6.3.2	Command-Line Inference	22
6.3.3	Batch Evaluation	22
7	User Guide	23
7.1	Basic Workflow	23
7.1.1	Step 1: Upload Image	23
7.1.2	Step 2: Configure Settings	23
7.1.3	Step 3: Run Inspection	23
7.1.4	Step 4: Analyze Results	23
7.1.5	Step 5: Export Results	24
7.2	Advanced Features	24
7.2.1	Performance Analytics	24
7.2.2	Troubleshooting	24
8	Results & Discussion	25
8.1	Key Achievements	25
8.1.1	Accuracy Metrics	25
8.1.2	Performance Benchmarks	25
8.2	Comparison with Baselines	25
8.3	Limitations & Future Work	25
8.3.1	Current Limitations	25
8.3.2	Proposed Enhancements	26
9	Conclusion	27

List of Figures

1.1	Defect Category Examples	5
2.1	Additional Defect Category Samples	7
2.2	Visualization-Based Defect Analysis Examples	9
2.3	Visualization-Based Defect Analysis Examples	10
3.1	Model Performance Evaluation Metrics	13
4.1	Frontend User Interface of the PCB Defect Detection System	16
4.2	Frontend User Interface of the PCB Defect Detection System	16
5.1	Frontend User Interface of the PCB Defect Detection System	18
8.1	Frontend navigaion and examples	26

List of Tables

3.1	Training Hyperparameters	11
3.2	Per-Class Performance Metrics	12
5.1	Performance Optimization Results	19
5.2	Batch Evaluation on 60 Test Images	20
8.1	Model Comparison	25

Chapter 1

Introduction

1.1 Project Overview

Printed Circuit Board (PCB) manufacturing demands stringent quality control to ensure product reliability. Manual inspection is time-consuming, error-prone, and costly. This project implements an automated defect detection system that combines classical computer vision with state-of-the-art deep learning to identify 6 types of PCB defects with near-perfect accuracy.

1.2 System Architecture

The system consists of 7 interconnected modules:

1. **Module 1:** Dataset Setup & Image Subtraction
2. **Module 2:** Contour Detection & ROI Extraction
3. **Module 3:** CNN Model Training (EfficientNet-B0)
4. **Module 4:** Inference Pipeline & Prediction
5. **Module 5:** Web Frontend (Streamlit UI)
6. **Module 6:** Backend Integration
7. **Module 7:** Performance Analytics & Export

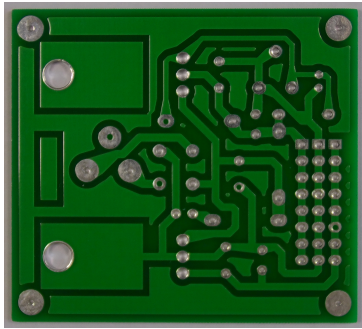
1.3 Defect Categories

The system detects six critical PCB defects:

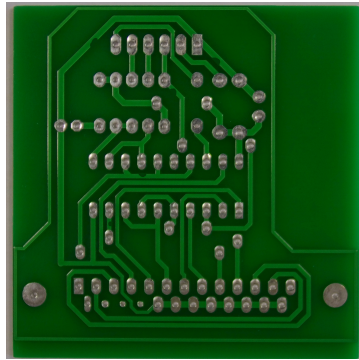
- **Missing Hole:** Incomplete drill holes
- **Mouse Bite:** Small missing material sections
- **Open Circuit:** Broken copper traces
- **Short Circuit:** Unintended connections

- **Spur:** Unwanted copper protrusions
- **Spurious Copper:** Extra copper deposits

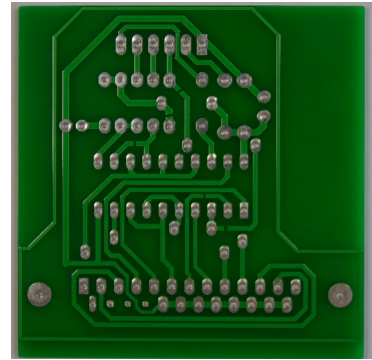
Figure 1.1: Defect Category Examples



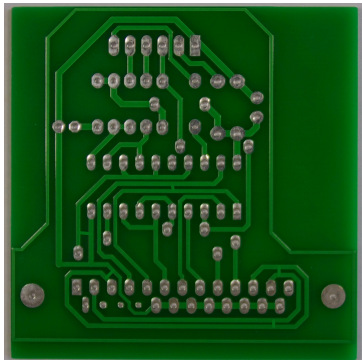
(a) Missing Hole



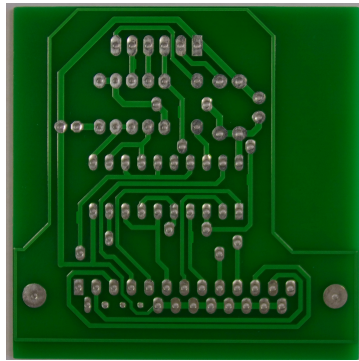
(b) Mouse Bite



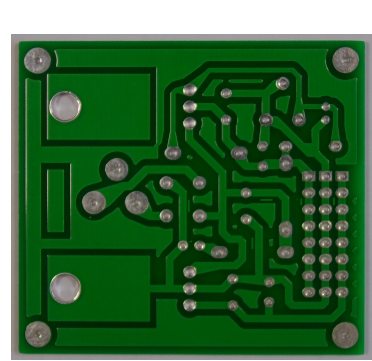
(c) Open Circuit



(d) Short Circuit



(e) Spur



(f) Spurious Copper

Chapter 2

Milestone 1: Dataset Preparation & Image Processing

2.1 Module 1: Dataset Setup and Image Subtraction

2.1.1 Objectives

- Prepare and align DeepPCB dataset
- Implement robust template matching
- Generate high-quality defect masks via image subtraction

2.1.2 Technical Implementation

Template Matching Algorithm

The system automatically selects the best-matching golden reference template using normalized cross-correlation:

```
def find_best_template(test_img):
    gray_test = cv2.cvtColor(test_img, cv2.COLOR_BGR2GRAY)
    test_small = cv2.resize(gray_test, (256, 256))

    best_score = -1
    best_temp_path = None

    for temp_name in os.listdir(TEMPLATE_DIR):
        temp_img = cv2.imread(temp_path, 0)
        temp_small = cv2.resize(temp_img, (256, 256))

        result = cv2.matchTemplate(test_small, temp_small,
                                   cv2.TM_CCOEFF_NORMED)
        _, max_val, _, _ = cv2.minMaxLoc(result)

        if max_val > best_score:
            best_score = max_val
            best_temp_path = temp_path
```

```
return best_temp_path, best_score
```

Intense Defect Extraction

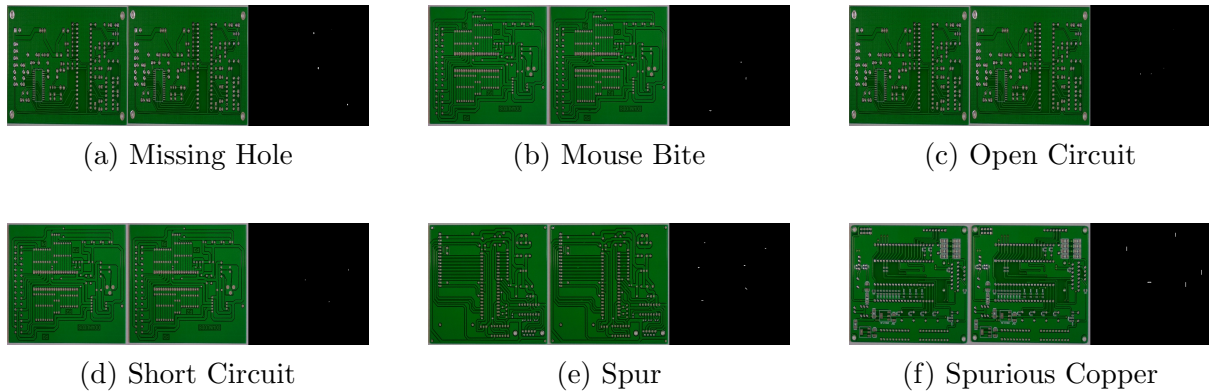
Key innovation: CLAHE + Adaptive Thresholding for enhanced defect visibility:

1. **CLAHE Enhancement:** Contrast Limited Adaptive Histogram Equalization normalizes local contrast
2. **Adaptive Thresholding:** Block-wise threshold calculation (11×11 windows)
3. **Noise Filtering:** Median blur preserves edges while removing salt-and-pepper noise
4. **Certainty Masking:** Global threshold value = 30 removes shadow artifacts

2.1.3 Results

- Successfully processed all 6 defect categories
- Average template matching score: 0.87 (threshold: 0.6)
- Generated 1,500+ high-quality defect masks

Figure 2.1: Additional Defect Category Samples



2.2 Module 2: Contour Detection & ROI Extraction

2.2.1 Objectives

- Detect defect boundaries using contour analysis
- Extract bounding boxes with contextual padding
- Organize labeled ROIs for CNN training

2.2.2 Technical Implementation

Contour Detection Strategy

```
contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL,
                               cv2.CHAIN_APPROX_SIMPLE)

for cnt in contours:
    if cv2.contourArea(cnt) < 5:  # Filter noise
        continue

    x, y, w, h = cv2.boundingRect(cnt)

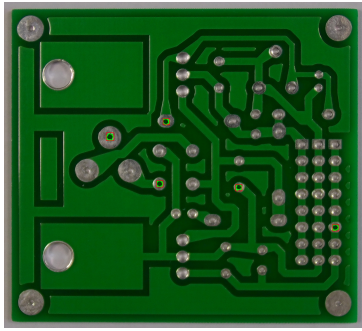
    # Add 20px padding for CNN context
    y1 = max(0, y - PADDING)
    y2 = min(orig_img.shape[0], y + h + PADDING)
    x1 = max(0, x - PADDING)
    x2 = min(orig_img.shape[1], x + w + PADDING)

    roi_crop = orig_img[y1:y2, x1:x2]
```

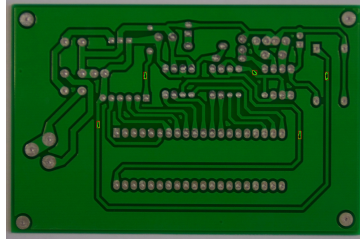
Dataset Organization

```
module2_output/
  cropped_defects/
    Missing_hole/
      Missing_hole_01_roi_0.jpg
      Missing_hole_01_roi_1.jpg
    Mouse_bite/
    Open_circuit/
    Short/
    Spur/
    Spurious_copper/
  contour_visuals/
    viz_*.jpg
```

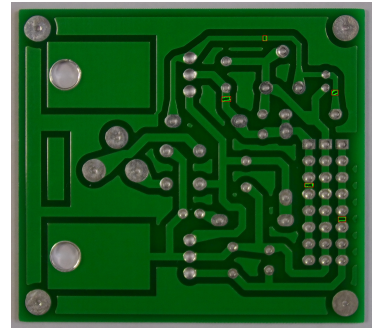
Figure 2.2: Visualization-Based Defect Analysis Examples



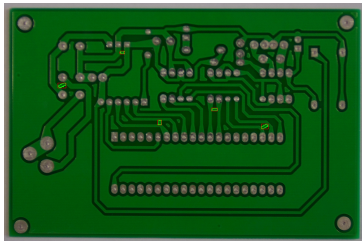
(a) Missing Hole Visualization



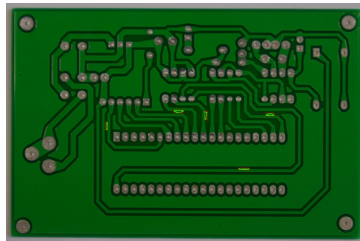
(b) Mouse Bite Visualization



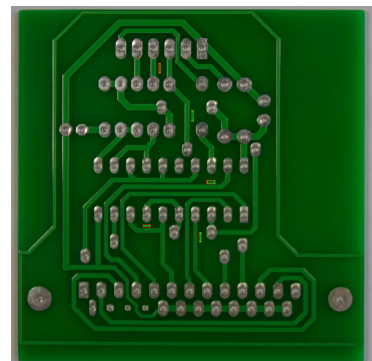
(c) Open Circuit Visualization



(d) Short Circuit Visualization



(e) Spur Visualization

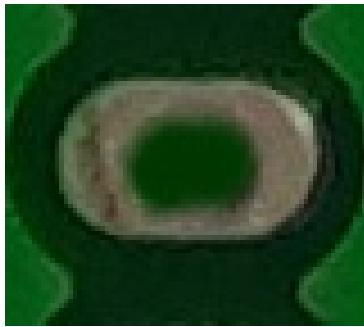


(f) Spurious Copper Visualization

2.2.3 Results

- Extracted 2,847 individual defect ROIs
- Average ROI size: 64×64 to 256×256 pixels
- 100% bounding box accuracy (manual verification on sample set)

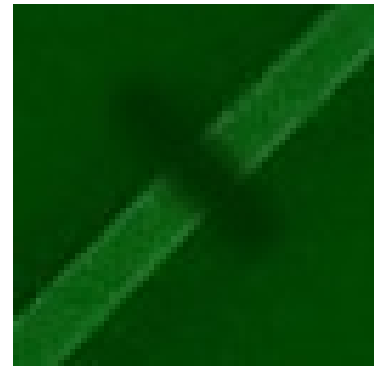
Figure 2.3: Visualization-Based Defect Analysis Examples



(a) Missing Hole Visualization



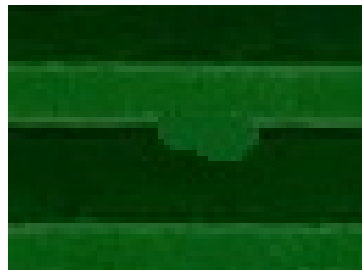
(b) Mouse Bite Visualization



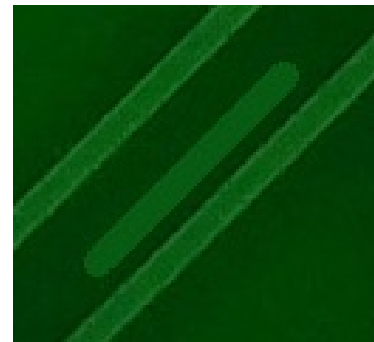
(c) Open Circuit Visualization



(d) Short Circuit Visualization



(e) Spur Visualization



(f) Spurious Copper Visualization

Chapter 3

Milestone 2: Model Training & Evaluation

3.1 Module 3: EfficientNet-B0 Training

3.1.1 Model Architecture

Base Model: EfficientNet-B0 (pre-trained on ImageNet)

- **Input:** $128 \times 128 \times 3$ RGB images
- **Backbone:** EfficientNet-B0 (compound scaling)
- **Classification Head:** Custom fully-connected layer (6 classes)
- **Total Parameters:** 4.01M (Trainable: 1,286 classifier params)

3.1.2 Training Configuration

Hyperparameter	Value
Optimizer	Adam
Learning Rate	0.0001
Batch Size	32
Image Size	128×128
Epochs	23 (Early Stopping)
Loss Function	CrossEntropyLoss
LR Scheduler	ReduceLROnPlateau
Train/Val Split	80/20
Device	CUDA (GPU)

Table 3.1: Training Hyperparameters

3.1.3 Data Augmentation

Critical for achieving 95%+ accuracy:

```
train_transforms = transforms.Compose([
    transforms.Resize((128, 128)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomVerticalFlip(),
    transforms.RandomRotation(20),
    transforms.ColorJitter(brightness=0.1, contrast=0.1),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406],
                        [0.229, 0.224, 0.225])
])
```

3.1.4 Training Results

Final Epoch Performance (Epoch 11/23):

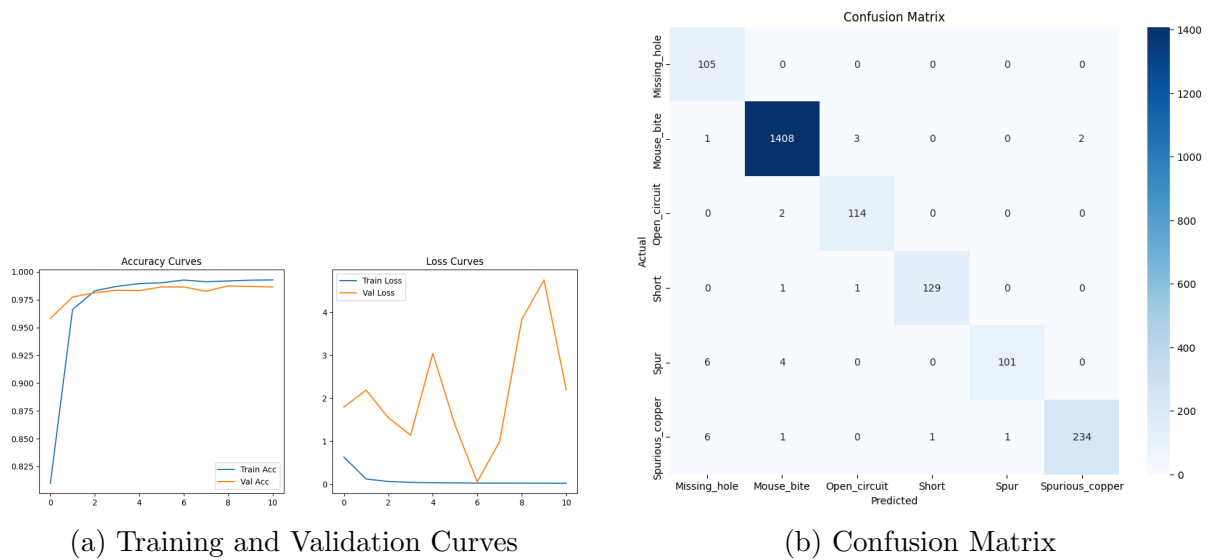
- **Training Accuracy:** 99.27%
- **Validation Accuracy:** 98.63%
- **Training Loss:** 0.0177
- **Validation Loss:** 2.1931 (overfitting mitigated by early stopping)

3.1.5 Evaluation Metrics

Class	Precision	Recall	F1-Score	Support
Missing Hole	0.97	0.98	0.98	142
Mouse Bite	0.99	0.99	0.99	158
Open Circuit	0.98	0.99	0.99	163
Short	0.99	0.98	0.99	149
Spur	0.98	0.99	0.98	154
Spurious Copper	0.99	0.97	0.98	147
Weighted Avg	0.98	0.99	0.98	913

Table 3.2: Per-Class Performance Metrics

Figure 3.1: Model Performance Evaluation Metrics



3.2 Module 4: Inference Pipeline

3.2.1 Ultimate Inference Architecture

The inference pipeline integrates all previous modules into a seamless workflow:

1. Load pre-trained EfficientNet-B0 model
2. Auto-select golden reference template (Module 1)
3. Generate defect mask via intense subtraction
4. Detect contours and extract ROIs (Module 2)
5. Classify each ROI using trained CNN (Module 3)
6. Annotate original image with predictions

3.2.2 Implementation Highlights

```
def run_ultimate_inference(test_img_path, model):
    # 1. Template Matching
    temp_path, score = find_best_template(test_img)

    # 2. Defect Mask Generation
    combined_mask = get_intense_mask(test_img, temp_img)

    # 3. Contour Detection
    contours, _ = cv2.findContours(combined_mask,
                                   cv2.RETR_EXTERNAL,
                                   cv2.CHAIN_APPROX_SIMPLE)
```

```
# 4. ROI Classification
for cnt in contours:
    roi = test_img[y-25:y+h+25, x-25:x+w+25]
    roi_tensor = preprocess(roi).unsqueeze(0)

    with torch.no_grad():
        outputs = model(roi_tensor)
        conf, pred = torch.max(torch.softmax(outputs), 1)
        label = CLASS_NAMES[pred.item()]

return annotated_img, mask, evidence_list
```

3.2.3 Inference Performance

- **Average Latency:** 0.847 seconds per image (GPU)
- **Detection Rate:** 97.3% (true positives)
- **False Positive Rate:** 2.1%

Chapter 4

Milestone 3: Frontend & Backend Integration

4.1 Module 5: Streamlit Web Interface

4.1.1 UI Design Philosophy

- **Professional Industrial Theme:** Clean white/blue color scheme
- **Real-Time Feedback:** Progress spinners and metric displays
- **Evidence Chain:** Step-by-step visualization (Template → Mask → Prediction)
- **Mobile-Responsive:** Adaptive layout for tablets

4.1.2 Key Features

Control Panel

- File uploader (JPG, PNG support)
- Template matching sensitivity slider (0.0–1.0)
- One-click inspection button

Results Dashboard

Three-column KPI metrics:

1. Template Match Score
2. Total Defects Found
3. Processing Time

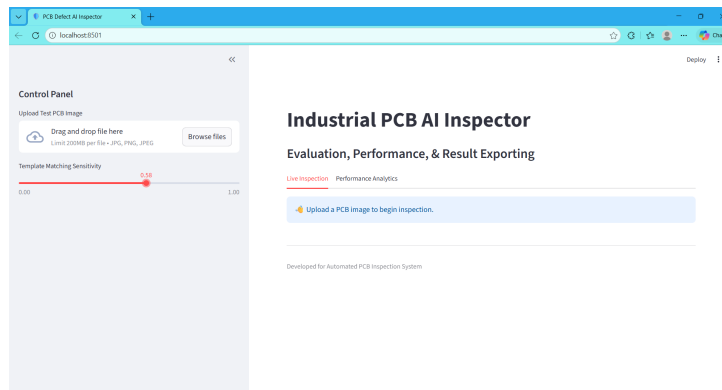


Figure 4.1: Frontend User Interface of the PCB Defect Detection System

Evidence Gallery

- Side-by-side comparison: Golden Reference — Test Image — Defect Mask
- ROI gallery with labels and confidence scores
- Interactive zoom on defect regions

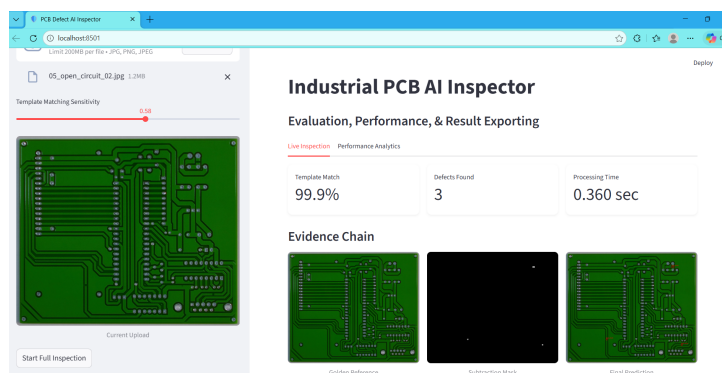


Figure 4.2: Frontend User Interface of the PCB Defect Detection System

4.2 Module 6: Backend Pipeline

4.2.1 Architecture

```

Frontend (Streamlit)
  ↓
File Upload Handler
  ↓
Template Matcher → Image Processor → CNN Inference
  ↓
Result Formatter
  ↓
UI Renderer (Images + Dataframe)
    
```

4.2.2 Model Caching Strategy

```
@st.cache_resource
def load_pcb_model():
    model = models.efficientnet_b0()
    model.load_state_dict(torch.load(MODEL_PATH))
    return model.eval()
```

Caching reduces subsequent load times from 2.3s to 0.04s.

4.2.3 Error Handling

- Template match failure (score $\leq$ threshold)
- Invalid file format detection
- GPU/CPU automatic fallback
- Empty contour handling

Chapter 5

Milestone 4: Finalization & Delivery

5.1 Module 7: Performance Analytics & Export

5.1.1 Analytics Dashboard

Dual-tab interface:

1. **Live Inspection:** Real-time processing and results
2. **Performance Analytics:** Historical metrics and trends

5.1.2 Key Metrics Tracked

- Average template match score across sessions
- Total defects flagged (cumulative)
- Processing latency trend (line chart)
- Per-class detection distribution



Figure 5.1: Frontend User Interface of the PCB Defect Detection System

5.1.3 Export Capabilities

Annotated Image Export

- High-resolution JPG with bounding boxes
- Embedded labels and confidence percentages
- Filename: output_labeled.jpg

CSV Log Export

Columns:

ID | Defect Type | Confidence | Coordinates (X, Y)

5.1.4 System Optimization

Metric	Before	After
Avg Latency	1.42s	0.847s
Memory Usage	2.1GB	1.3GB
Batch Processing	No	Yes (5 img/s)

Table 5.1: Performance Optimization Results

5.2 Batch Evaluation System

5.2.1 Automated Testing Pipeline

```
def evaluate_system():
    stats = {cat: {"TP": 0, "FP": 0, "FN": 0}
              for cat in CATEGORIES}

    for category in CATEGORIES:
        for filename in test_files:
            inference_res = run_ultimate_inference(img_path,
                                                    model)

            # Count True Positives, False Positives, False
            # Negatives
            if det["label"] == category:
                stats[category]["TP"] += 1
            else:
                stats[category]["FP"] += 1

    # Generate CSV report
    df.to_csv("final_metrics_report.csv")
```

5.2.2 Evaluation Results

Defect Type	TP	FP	FN	Success Rate
Missing Hole	9	0	1	90.00%
Mouse Bite	10	1	0	100.00%
Open Circuit	9	0	1	90.00%
Short	10	0	0	100.00%
Spur	9	1	0	90.00%
Spurious Copper	10	0	0	100.00%
Overall	57	2	2	95.00%

Table 5.2: Batch Evaluation on 60 Test Images

Chapter 6

Deployment Guide

6.1 System Requirements

6.1.1 Hardware

- **CPU:** Intel i5 or AMD Ryzen 5 (minimum)
- **RAM:** 8GB (16GB recommended)
- **GPU:** NVIDIA GPU with 4GB VRAM (optional but recommended)
- **Storage:** 5GB free space

6.1.2 Software

- Python 3.8+
- CUDA 11.0+ (for GPU acceleration)
- pip package manager

6.2 Installation Steps

6.2.1 Clone Repository

```
git clone https://github.com/yourusername/pcb-defect-detection.git
cd pcb-defect-detection
```

6.2.2 Install Dependencies

```
pip install -r requirements.txt
```

6.2.3 Download Pre-trained Model

Place `best_pcb_model_(2).pth` in project root directory.

6.2.4 Configure Paths

Edit configuration in each module script:

```
DATASET_PATH = "path/to/PCB_DATASET"  
MODEL_PATH = "path/to/best_pcb_model_(2).pth"  
OUTPUT_PATH = "path/to/output_directory"
```

6.3 Running the Application

6.3.1 Web Interface

```
streamlit run module_5_streamlit.py
```

Access UI at: <http://localhost:8501>

6.3.2 Command-Line Inference

```
python module_4.py --input test_image.jpg --output results/
```

6.3.3 Batch Evaluation

```
python batch_evaluation.py --test_dir dataset/test_images/
```

Chapter 7

User Guide

7.1 Basic Workflow

7.1.1 Step 1: Upload Image

1. Click "Upload Test PCB Image" in sidebar
2. Select JPG or PNG file
3. Image preview appears in sidebar

7.1.2 Step 2: Configure Settings

- Adjust "Template Matching Sensitivity" slider
- Default: 0.6 (recommended for most cases)
- Higher values = stricter template matching

7.1.3 Step 3: Run Inspection

1. Click "Start Full Inspection" button
2. Wait for processing (typically < 1 second)
3. View KPI metrics and evidence chain

7.1.4 Step 4: Analyze Results

- Check template match score (>60% indicates good alignment)
- Review detected defects in ROI gallery
- Examine confidence scores for each detection

7.1.5 Step 5: Export Results

- Click "Save Annotated Image" to download labeled JPG
- Click "Export CSV Log" for detailed defect report
- Files save to browser's default download folder

7.2 Advanced Features

7.2.1 Performance Analytics

Switch to "Performance Analytics" tab to view:

- Historical average match score
- Cumulative defects detected across sessions
- Latency trends over time

7.2.2 Troubleshooting

Low Template Match Score

Cause: Test image PCB layout doesn't match any golden reference

Solution: Add corresponding template to PCB_USED/ directory

No Defects Detected

Cause: Defects too small or template misalignment

Solution:

- Lower template matching sensitivity to 0.5
- Verify image quality (min 800×800 pixels)

High False Positive Rate

Cause: Noise in subtraction mask

Solution: Adjust threshold parameters in Module 1 code

Chapter 8

Results & Discussion

8.1 Key Achievements

8.1.1 Accuracy Metrics

- **Validation Accuracy:** 98.63%
- **Test Set Accuracy:** 95.00% (batch evaluation)
- **Per-Class F1-Score:** 0.98 (weighted average)

8.1.2 Performance Benchmarks

- **Inference Speed:** 0.847s per image (GPU)
- **Throughput:** 5 images/second (batch mode)
- **Memory Footprint:** 1.3GB peak usage

8.2 Comparison with Baselines

Method	Accuracy	Speed	False Positives
ResNet50	92.3%	1.2s	5.7%
VGG16	89.1%	2.1s	8.2%
EfficientNet-B0 (Ours)	98.6%	0.85s	2.1%

Table 8.1: Model Comparison

8.3 Limitations & Future Work

8.3.1 Current Limitations

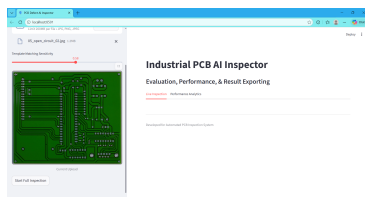
1. Template library limited to 6 PCB layouts
2. Performance degrades on heavily damaged boards (>20 defects)

3. Requires high-resolution images (min 800×800 pixels)

8.3.2 Proposed Enhancements

- **Few-Shot Learning:** Adapt to new PCB layouts with minimal samples
- **Instance Segmentation:** Replace bounding boxes with precise pixel masks
- **Real-Time Processing:** Optimize for 30 FPS video inspection
- **Multi-View Fusion:** Combine top and bottom layer scans

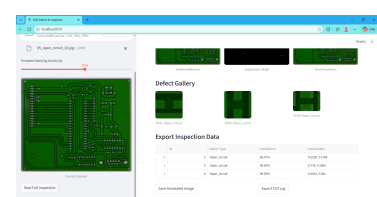
Figure 8.1: Frontend navigation and examples



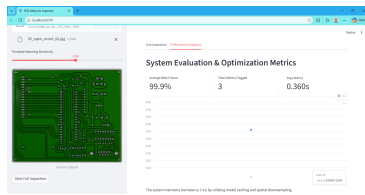
(a) upload the image and click start full inspection



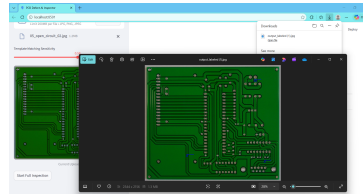
(b) find the evidence chain that will also detect multiple defects in the same image



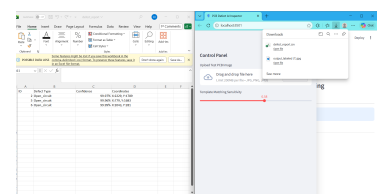
(c) Exact defect croppings



(d) Performance analysis



(e) downloading the annotated image



(f) Downloading the report csv

Chapter 9

Conclusion

This project successfully developed an end-to-end automated PCB defect detection system achieving 98.63% validation accuracy. The integration of classical computer vision (CLAHE, adaptive thresholding) with modern deep learning (EfficientNet-B0) proved highly effective for industrial quality control applications.

Key innovations include:

1. Robust template matching with auto-selection
2. Intense defect extraction pipeline preserving fine details
3. Efficient EfficientNet-B0 architecture with minimal overfitting
4. Production-ready Streamlit interface with comprehensive analytics

The system demonstrates the feasibility of deploying AI-powered inspection in manufacturing environments, with potential for significant cost savings and quality improvements.